

Sesión de Prácticas

Make Classic Games

¿Prácticas?

En las sesiones prácticas, vamos a crear un juego desde cero; a través de una serie de retos o pasos para crear dicho juego.

Cada semana, se resolverá la semana anterior, y se propondrá un nuevo reto.

Proyecto

Nuestro primer proyecto será....



Un Shot'emup realizado con Raylib.

¿Raylib?

Raylib, es una librería que permite crear juegos, herramientas, aplicaciones de forma programática; utilizando principalmente el lenguaje C.

- No tiene dependencias externas*.
- Api Fácil de utilizar.
- Dispone de diferentes Bindings para muchos lenguajes.
- Es Multiplataforma.
- Es “multi consola” (Versiones para diferentes consolas).

<https://www.raylib.com/>

* Puede requerir de dependencias como OpenGL o SDL dependiendo del sistema y/o entorno utilizado.

Primer reto.. Instalar raylib

Raylib, es multiplataforma; por lo que podemos instalarlo en diferentes entornos:

- Windows

<https://github.com/raysan5/raylib/wiki/Working-on-Windows>

<https://raysan5.itch.io/raylib>

- Linux

<https://github.com/raysan5/raylib/wiki/Working-on-GNU-Linux>

- MacOS

<https://github.com/raysan5/raylib/wiki/Working-on-macOS>

Segundo reto... Crear proyecto para Raylib

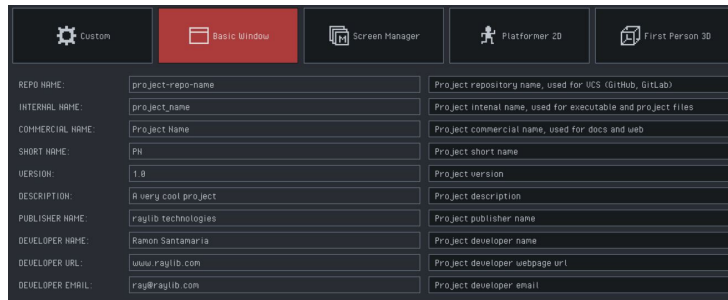
El segundo reto es compilar y ejecutar un proyecto para Raylib

Se deberá crear un proyecto básico y ejecutarlo en nuestra máquina.

Puedes usar la herramienta “Raylib Project Creator” para generar tu proyecto:

<https://raysan5.itch.io/raylib-project-creator>

<https://github.com/raylib-extras/raylib-quickstart>



The screenshot shows the Raylib Project Creator interface with the 'Basic Window' template selected. The interface is divided into a top navigation bar and a main configuration area. The navigation bar includes tabs for 'Custom', 'Basic Window' (selected), 'Screen Manager', 'Platformer 2D', and 'First Person 3D'. The main configuration area contains a table of fields for project settings, each with a label, a text input field, and a description.

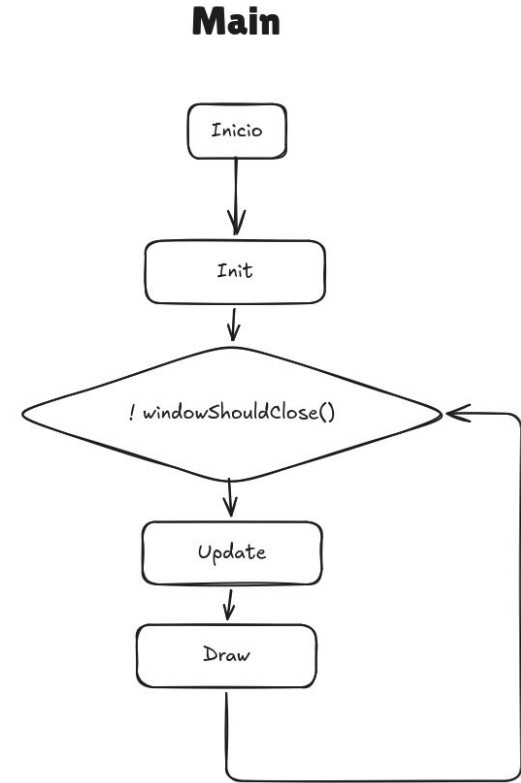
Field	Value	Description
REPO NAME:	project-repo-name	Project repository name, used for VCS (Github, GitLab)
INTERNAL NAME:	project_name	Project internal name, used for executable and project files
COMMERCIAL NAME:	Project Name	Project commercial name, used for docs and web
SHORT NAME:	PN	Project short name
VERSION:	1.0	Project version
DESCRIPTION:	A very cool project	Project description
PUBLISHER NAME:	raylib technologies	Project publisher name
DEVELOPER NAME:	Ramon Santanaria	Project developer name
DEVELOPER URL:	www.raylib.com	Project developer webpage url
DEVELOPER EMAIL:	ray@raylib.com	Project developer email

Tercer Reto Init, Update y Draw

Vamos a estructurar nuestro código de la siguiente forma:

- La función Main que será la principal; que contendrá llamadas a las siguientes funciones:
 - *Init*: Inicializa las variables necesarias.
 - *Update*: Actualiza la lógica de nuestro juego.
 - *Draw*: Dibuja el estado actual de nuestro juego.

NOTA: Las funciones Update y Draw pueden ser llamadas también desde la función *UpdateDrawFrame()*.



Texturas

Una “Textura” en Raylib, corresponde a un elemento cargado en la VRAM (Memoria de la tarjeta gráfica).

En Raylib, podemos tratarlas como textura en 2D o como una textura para aplicar a un modelo,etc...

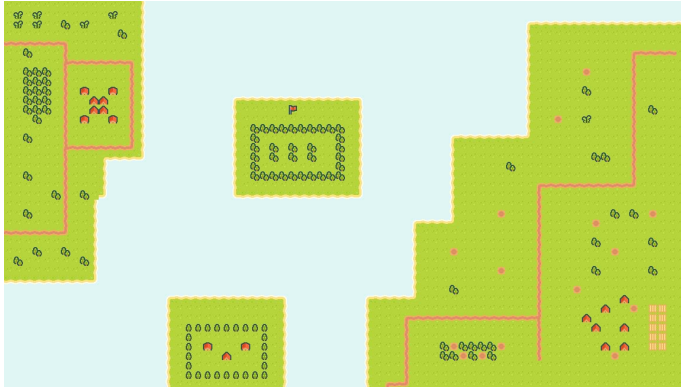
Con la función *LoadTexture2D*, podemos cargar una imagen como una textura.

NOTA: ES importante también descargar la textura de la VRAM con la función *UnloadTexture*.

Recursos

Puedes descargar los recursos a utilizar en este proyecto en la siguiente dirección:

https://makeclassicgames.dev/godotosc2025/assets/shotemup_resources.zip



Cuarto Reto... Jugador

Vamos a crear nuestro jugador; para ello definiremos las siguientes propiedades:

- Position (*Vector2D*).
- Velocidad (*Vector2D*).
- Textura (*Texture2D*).



Y crearemos las siguientes funciones:

- *InitPlayer*(Player *,x,y); Inicializa un jugador en una posición y carga las texturas.
- *UpdatePlayer*(Player*): Actualiza la posición en función de la entrada.
- *DrawPlayer*(Player*): Dibuja un jugador dependiendo del estado actual.
- *UnloadPlayer*(Player*): Limpia la memoria de texturas/ memoria dinámica utilizada.

Referencias

- Raylib: <https://www.raylib.com/>
- Tutorial de C: <https://slides.com/zerasul/introduccion-a-c>